Solve Sudoku

AlgoExpert · Hard · Recursion

The problem

- Fill a 9×9 board (0 = empty) in place via backtracking with row/col/box bitmasks.

Intuition

- Backtracking with bitmasks of used digits per row, column and box makes each check constant time.

Approach

- 1. Record used digits
- 2. Find the next empty cell
- 3. Try each allowed digit and recurse
- 4. Undo on failure

Java solution (1/2)

```java
public class SolveSudoku {
    public static int[][] solveSudoku(int[][] b) {
        int[] rows = new int[9], cols = new int[9], boxes = new int[9];
        for (int r = 0; r < 9; r++) for (int c = 0; c < 9; c++) if (b[r][c] != 0) {
            int bit = 1 << b[r][c];
            rows[r] |= bit; cols[c] |= bit; boxes[r / 3 * 3 + c / 3] |= bit;
        }
        solve(b, 0, rows, cols, boxes);
        return b;
    }
```

Java solution (2/2)

```
    private static boolean solve(int[][] b, int pos, int[] rows, int[] cols, int[] boxes) {
        if (pos == 81) return true;
        int r = pos / 9, c = pos % 9, bx = r / 3 * 3 + c / 3;
        if (b[r][c] != 0) return solve(b, pos + 1, rows, cols, boxes);
        for (int d = 1; d <= 9; d++) {
            int bit = 1 << d;
            if (((rows[r] | cols[c] | boxes[bx]) & bit) != 0) continue;
            b[r][c] = d; rows[r] |= bit; cols[c] |= bit; boxes[bx] |= bit;
            if (solve(b, pos + 1, rows, cols, boxes)) return true;
            b[r][c] = 0; rows[r] ^= bit; cols[c] ^= bit; boxes[bx] ^= bit;
        }
        return false;
    }
}
```

Complexity

- Time: O(9^empty) worst
- Space: O(1) besides recursion

Recap

- Backtracking with bitmasks of used digits per row, column and box makes each check constant time.
- Complexity: O(9^empty) worst